

Exercício em duplas: qual é a saída?

Paradigmas de Linguagens de Programação · Exemplos 07_exercicio

1 · Python

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

(a) Saída:

```
[1]  
[1, 2]
```

(b) Conceito:

Argumento padrão mutável (avaliado uma única vez). O valor padrão [] é criado quando a função é *definida*, não a cada chamada. Assim, todas as chamadas sem o segundo argumento compartilham a mesma lista, que acumula os itens. Correção usual: lista=None e criar a lista dentro da função.

2 · Java

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
  
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

(a) Saída:

```
0 5
```

(b) Conceito:

Passagem de parâmetros por valor. Java sempre passa uma cópia do argumento. Para o int n, a cópia é alterada e o original permanece 5. Para o array, a cópia é do *valor da referência*: ambas apontam para o mesmo objeto, então v[0] = 0 altera o array original.

3 · Python

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

(a) Saída:

[2, 2, 2]

(b) Conceito:

Closures e vinculação tardia (late binding). As lambdas capturam a *variável* `i`, não o seu valor no momento da criação. Quando são chamadas, o laço já terminou e `i` vale 2. Correção usual:

`lambda i=i: i.`

4 · C

```
int contador(void) {
    static int n = 0;
    return ++n;
}

// em main:
contador(); contador();
printf("%d\n", contador());
```

(a) Saída:

3

(b) Conceito:

Variável local estática (duração de armazenamento estática). `static int n` é inicializada uma única vez e persiste entre as chamadas, embora seu escopo seja local à função. As três chamadas retornam 1, 2 e 3.

5 · Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}

let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

(a) Saída:

Erro de compilação (E0382): borrow of moved value: `v`. Nada é impresso.

(b) Conceito:

Ownership e movimentação (move semantics). Ao chamar `dobra(v)`, a posse do `Vec` é movida para a função e `v` deixa de ser válida em `main`. Usá-la no `println!` viola a regra de posse única. Correções: passar `&v` (empréstimo) ou `v.clone()`.

6 · Python

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

(a) Saída:

UnboundLocalError: cannot access local variable 'total' where it is not associated with a value

(b) Conceito:

Escopo de variáveis (regra LEGB). Como há atribuição a `total` dentro da função, o Python a trata como variável *local* em todo o corpo. Ao avaliar `total + x`, a local ainda não tem valor. Correção: declarar `global total` (ou, preferencialmente, receber e retornar o valor).

Resumo

#	Linguagem	Saída	Conceito
1	Python	[1] e [1, 2]	Argumento padrão mutável
2	Java	0 5	Passagem por valor (referência copiada)
3	Python	[2, 2, 2]	Closure / late binding
4	C	3	Variável local static
5	Rust	Erro de compilação	Ownership / move
6	Python	UnboundLocalError	Escopo (LEGB)